Python Notes (Lessons 1–12)

1. Variables

Definition

A variable is a container used to store data.

```
name = "Vipin"
age = 25
```

---

2. Data Types

String (str)

Stores text.

```
name = "Vipin"
```

Integer (int)

Stores whole numbers.

```
age = 25
```

Float (float)

Stores decimal numbers.

```
price = 99.99
```

Boolean (bool)

Stores True or False.

```
is_logged_in = True
```

---

3. Type Checking

Syntax

```
type(variable)
```

Example

```
x = 10
print(type(x))
```

Output:

```
<class 'int'>
```

---

4. Type Casting

Definition

Converting one data type into another.

Examples

```
int("10")
float("10")
str(10)
```

Common Usage

```
num = int(input("Enter a number: "))
```

---

5. Input and Output

Input

```
name = input("Enter your name: ")
```

Output

```
print(name)
```

f-string

```
name = "Vipin"
age = 25

print(f"My name is {name} and I am {age} years old")
```

Operators

6. Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Division
%	Modulus
**	Power

Examples

```
10 + 5
10 - 5
10 * 5
10 / 5
10 // 3
10 % 3
10 ** 2
```

---

7. Comparison Operators

Operator	Meaning
==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than Equal
<=	Less Than Equal

Example

```
10 > 5
```

Output:

```
True
```

---

8. Logical Operators

and

```
age >= 18 and has_id == "yes"
```

Both conditions must be True.

or

```
marks >= 90 or sports_quota == "yes"
```

Any one condition can be True.

not

```
not True
```

Output:

```
False
```

---

9. Ternary Operator

Definition

Short form of if-else.

Syntax

```
value_if_true if condition else value_if_false
```

Example

```
age = 20

result = "Adult" if age >= 18 else "Minor"

print(result)
```

Conditional Statements

10. if-else

Syntax

```
if condition:
    statement
else:
    statement
```

Example

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

---

11. Nested if

Example

```
if card_inserted == "yes":
    if pin == "1234":
        print("Access Granted")
```

Loops

12. for Loop

Syntax

```
for i in range(start, stop, step):
    print(i)
```

Example

```
for i in range(1, 6):
    print(i)
```

Common range() Usage

```
range(5)
range(1, 6)
range(1, 11, 2)
range(10, 0, -1)
```

---

13. while Loop

Syntax

```
while condition:
    statement
```

Example

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

---

14. break

Stops loop immediately.

```
for i in range(10):
    if i == 5:
        break
```

---

15. continue

Skips current iteration.

```
for i in range(10):
    if i == 5:
        continue
```

Strings

16. String Basics

Definition

A string is a collection of characters.

```
name = "Python"
```

---

17. String Traversal

Using for Loop

```
for ch in "Python":
    print(ch)
```

Using while Loop

```
text = "Python"

i = 0

while i < len(text):
    print(text[i])
    i += 1
```

---

18. len()

Returns length of a string.

```
len("Python")
```

Output:

```
6
```

---

19. Useful String Methods

Lowercase

```
text.lower()
```

Uppercase

```
text.upper()
```

Check Alphabet

```
text.isalpha()
```

Check Digit

```
text.isdigit()
```

Check Uppercase

```
text.isupper()
```

Check Lowercase

```
text.islower()
```

---

20. String Practice Problems Covered

- Count characters
- Count vowels
- Count uppercase letters
- Reverse string
- Palindrome string
- Remove spaces
- Print alternate characters

Nested Loops

21. Nested Loop Syntax

```
for i in range(rows):
    for j in range(cols):
        print("*")
```

Pattern Programming

Pattern 1

```
*
**
***
****
*****
```

```
for i in range(5):
    for j in range(i + 1):
        print("*", end="")
    print()
```

Pattern 2

```
1
12
123
1234
12345
```

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end="")
    print()
```

Pattern 3

```
*****
****
***
**
*
```

```
for i in range(5):
    for j in range(i, 5):
```

```
        print("*", end="")
    print()
```

Pattern 4

```
1
22
333
4444
55555
```

```
for i in range(1, 6):
    for j in range(i):
        print(i, end="")
    print()
```

Number Logic

22. Digit Extraction

Get Last Digit

```
digit = num % 10
```

Example

```
1234 % 10
```

Output:

```
4
```

---

23. Remove Last Digit

```
num = num // 10
```

Example

```
1234 // 10
```

Output:

```
123
```

---

24. Count Digits

```
num = 12345

count = 0

while num > 0:
    count += 1
    num = num // 10

print(count)
```

Output:

```
5
```

---

25. Sum of Digits

```
num = 1234

sum_digits = 0

while num > 0:
    digit = num % 10
    sum_digits += digit
    num = num // 10

print(sum_digits)
```

Output:

```
10
```

26. Reverse Number

```
num = 1234

rev = 0

while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num = num // 10

print(rev)
```

Output:

```
4321
```

27. Palindrome Number

Definition

A number that remains same after reversing.

Examples:

```
121
1331
12321
```

Program

```
num = 121

temp = num
rev = 0

while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num = num // 10

if temp == rev:
    print("Palindrome")
```

```
else:
    print("Not Palindrome")
```

Important Concepts Learned

✅ Variables

✅ Data Types

✅ Type Casting

✅ Input / Output

✅ Arithmetic Operators

✅ Comparison Operators

✅ Logical Operators

✅ Ternary Operator

✅ if-else

✅ Nested if

✅ for Loop

✅ while Loop

✅ break

✅ continue

✅ Strings

✅ String Traversal

✅ Nested Loops

✅ Pattern Programming

✅ Digit Extraction

✅ Count Digits

✅ Sum of Digits

✅ Reverse Number

✅ Palindrome Number

Current Level

You can now:

- Write basic Python programs
- Use conditions effectively
- Work with loops confidently
- Solve beginner logic problems
- Solve basic number-based DSA problems
- Work with strings
- Create simple pattern programs
- Debug simple syntax and logic errors

Current Stage:

```
Python Beginner → Beginner+
```

Next Topic:

```
Lesson 13: Lists
```